

EduGenie: AI-Powered Educational Learning Assistant

Project Description

EduGenie is an AI-powered educational learning assistant designed to enhance the learning experience of students through personalized and intelligent educational support. The platform leverages Google's Gemini AI to provide instant answers to academic questions, detailed concept explanations, automatic quiz generation, text summarization, and personalized learning guidance.

The application helps students learn more effectively by reducing the time spent searching for information and providing accurate, AI-generated educational content in real time. EduGenie is built using FastAPI, Python, HTML, CSS, and JavaScript, and is deployed on Render for public accessibility.

By integrating advanced Generative AI capabilities with an intuitive user interface, EduGenie empowers students to improve their understanding of subjects, strengthen their knowledge, and achieve better academic performance.

Scenarios

Scenario 1

A student preparing for semester examinations enters a difficult topic such as "Internet of Things Architecture." EduGenie instantly provides a detailed explanation, key concepts, and important points, helping the student understand the topic quickly and effectively.

Scenario 2

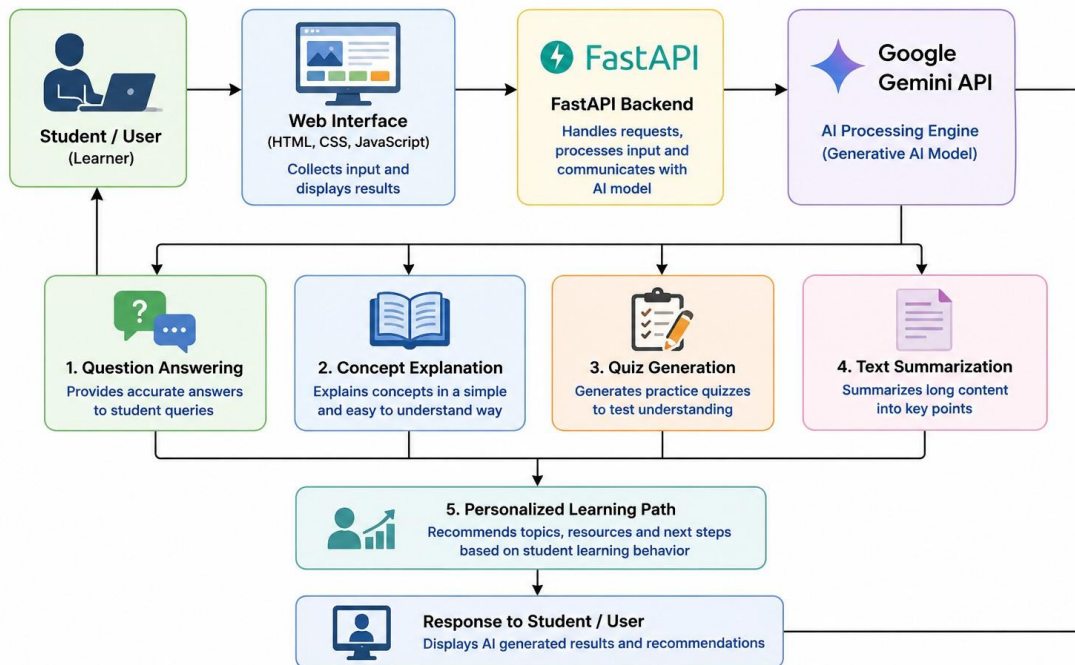
A learner studying a lengthy article or academic document uses the Text Summarization feature. EduGenie generates a concise summary highlighting the most important concepts, saving time and improving revision efficiency.

Scenario 3

A student wants to test their understanding of a subject. After entering a topic such as "Cloud Computing," EduGenie automatically generates a quiz with relevant questions, allowing the student to assess their knowledge and identify areas for improvement.

Technical Architecture:

EduGenie – Technical Architecture Diagram



Description

EduGenie follows a modular AI-powered architecture to provide personalized educational assistance. The frontend interface built using HTML, CSS, and JavaScript collects user queries and learning requests. The FastAPI backend processes these requests and securely communicates with Google's Gemini AI API to generate intelligent educational responses. The system provides features such as Question Answering, Concept Explanation, Quiz Generation, Text Summarization, and Personalized Learning Guidance. The application is deployed on Render, ensuring accessibility and scalability for learners.

Pre-requisites

- Python Programming Knowledge
- FastAPI Framework Knowledge
- Google Gemini API Familiarity
- HTML, CSS and JavaScript Skills
- Git & GitHub Version Control
- VS Code Development Environment
- Render Deployment Knowledge

Project Workflow:

Milestone 1: Requirement Analysis and Planning

- **Activity 1.1:** Identify learning challenges faced by students and define project objectives.
- **Activity 1.2:** Analyze AI-powered educational assistant requirements.
- **Activity 1.3:** Select Google Gemini AI as the core AI model.
- **Activity 1.4:** Define system architecture and technology stack.

Milestone 2: Backend Development

- **Activity 2.1:** Develop FastAPI backend for handling user requests.
- **Activity 2.2:** Integrate Google Gemini API for AI-powered responses.
- **Activity 2.3:** Implement Question Answering and Concept Explanation modules.

Milestone 3: Core Feature Development

- **Activity 3.1:** Develop Quiz Generation functionality.
- **Activity 3.2:** Implement Text Summarization module.
- **Activity 3.3:** Develop Personalized Learning Path recommendations.

Milestone 4: Frontend Development

- **Activity 4.1:** Design user interface using HTML, CSS, and JavaScript.
- **Activity 4.2:** Create responsive pages for user interaction.
- **Activity 4.3:** Connect frontend with FastAPI backend services.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Perform functional testing of all modules.
- **Activity 5.2:** Fix bugs and optimize performance.
- **Activity 5.3:** Deploy the application on Render.

Milestone 6: Project Completion and Documentation

- **Activity 6.1:** Prepare project documentation.
- **Activity 6.2:** Create GitHub repository and upload source code.
- **Activity 6.3:** Demonstrate the application and submit final deliverables.

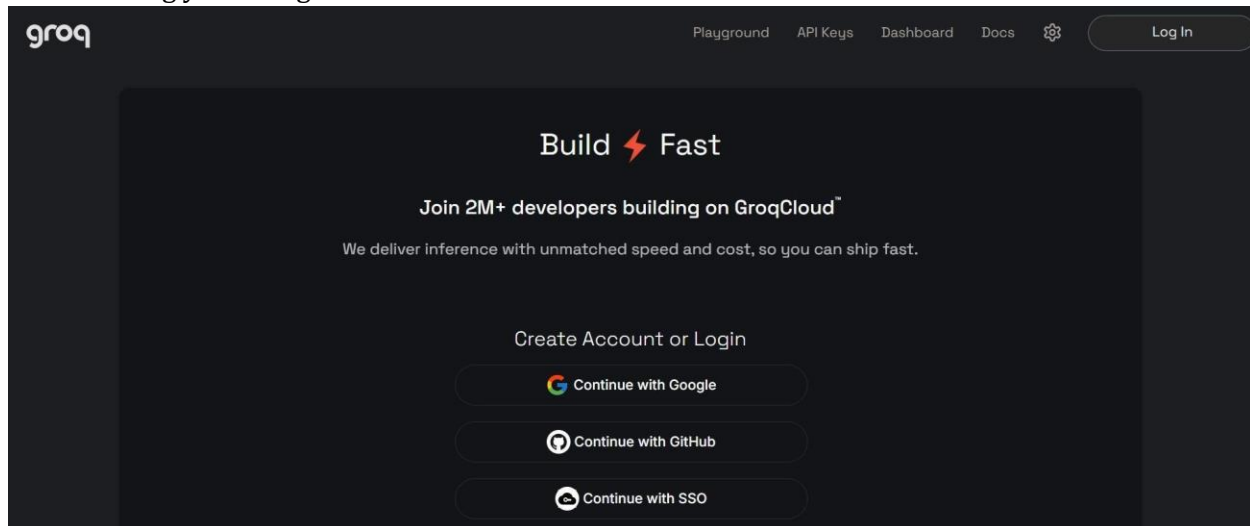
Milestone 1: Requirement Gathering and Analysis

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

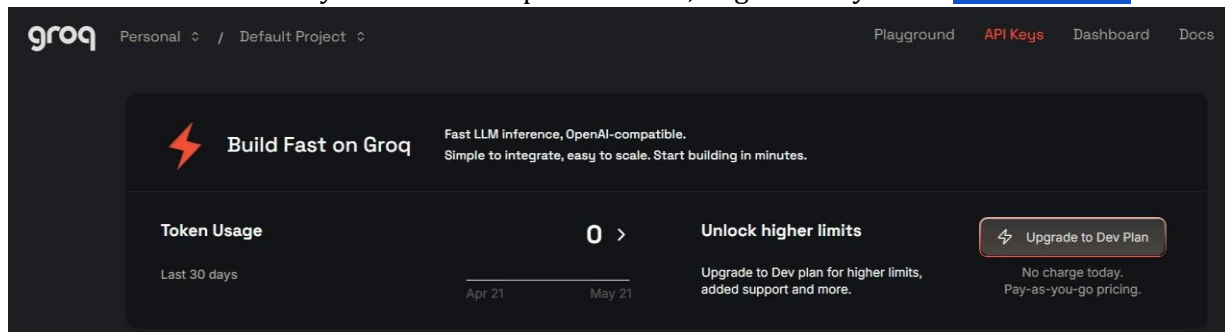
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

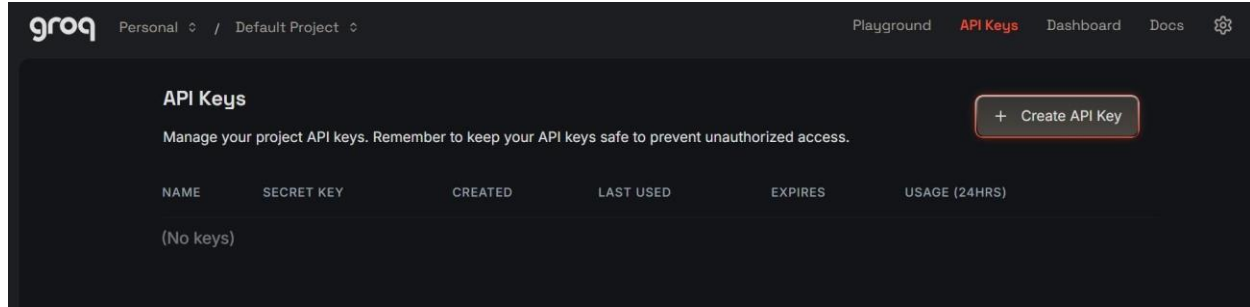
- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.



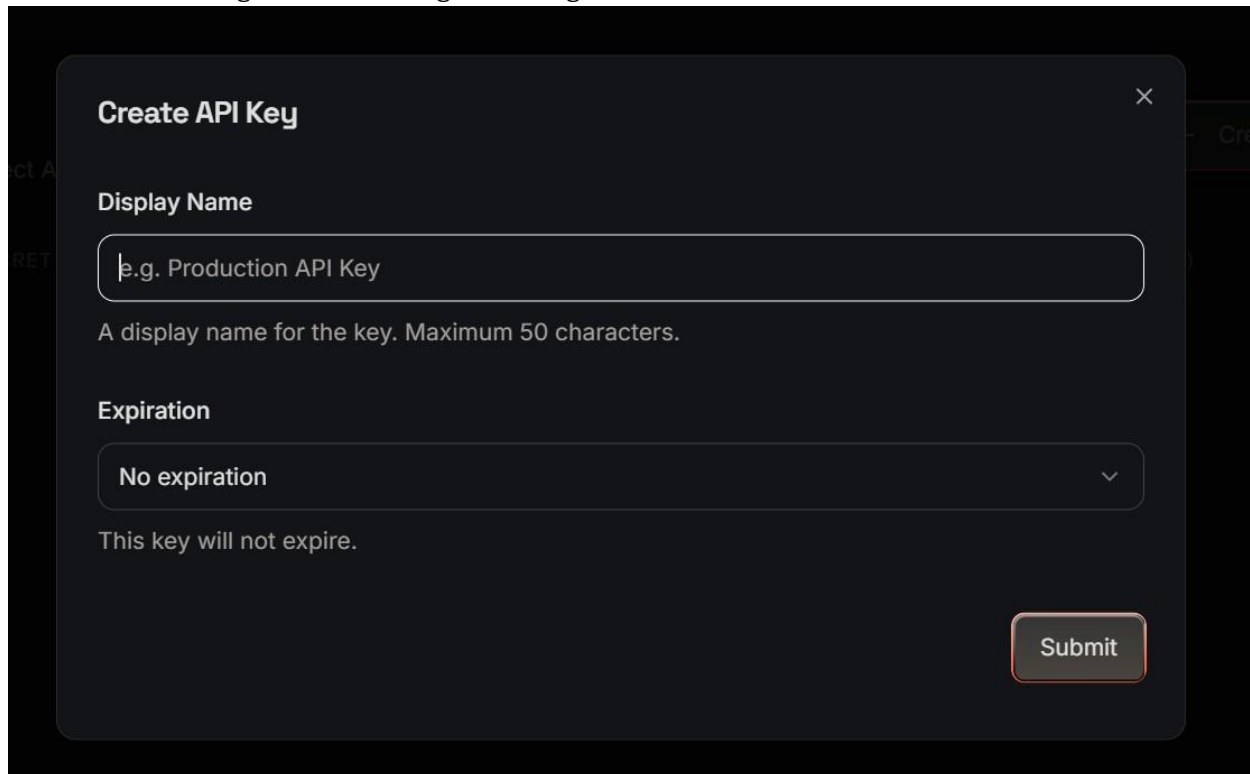
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the [API Keys page](#).



- **Step 3 — Create a New API Key:** Click the “Create API Key” button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later.



- **Step 4 — API Key:** Now give a “Display Name” and “Expiration” is optional, and Click on “Submit”. The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

GROQ_API_KEY=your_copied_api_key_here

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the CaptionAI application, focusing on the types of content it will generate (captions, hashtags, and calls-to-action across Instagram and LinkedIn).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to social media copywriting tasks.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of creative output quality and generation speed, set with a temperature of 0.8 and max_tokens of 2048 for rich, varied captions.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application entering product/campaign information, selecting a platform (Instagram or LinkedIn), and choosing a tone (Professional, Casual, or Promotional).
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using TONE_DESCRIPTIONS and PLATFORM_TIPS dictionaries, calls the Groq API, parses the JSON response via `extract_json()`, and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

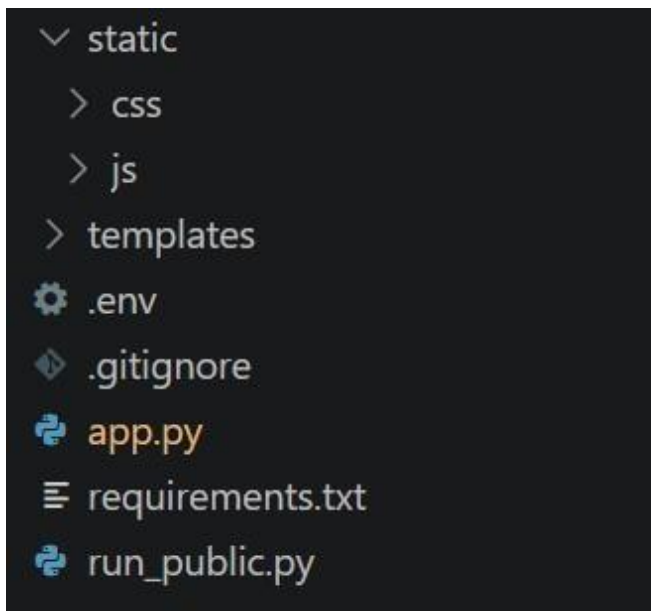
- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Learning Features

Implement the core educational features of EduGenie to provide intelligent learning assistance using Google Gemini AI.

- **Question Answering:** Provides accurate answers to academic questions submitted by students.
- **Concept Explanation:** Generates detailed and easy-to-understand explanations for learning topics.
- **Quiz Generation:** Creates topic-based quizzes to help students assess their understanding.
- **Text Summarization:** Converts lengthy educational content into concise summaries.
- **Personalized Learning Path:** Recommends learning resources and study paths based on user needs.

Activity 2.2: Implement the FastAPI Backend

Define API Routes:

- Create API endpoints to handle educational queries and learning requests.
- Configure routes for Question Answering, Quiz Generation, and Summarization services.

Process User Input:

- Collect user queries through the web interface.
- Validate user input before processing.
- Forward requests to the Gemini AI service.

Integrate AI Services:

- Connect FastAPI backend with Google Gemini API.
- Send prompts and receive AI-generated responses.
- Process and format responses before displaying them to users.

Response Management:

- Handle successful and failed API responses.
- Implement exception handling for improved reliability.
- Ensure smooth communication between frontend and backend services.

Milestone 3: App.py Development

This milestone focuses on developing the core backend logic of EduGenie using FastAPI. The backend handles user requests, communicates with Google Gemini AI, processes responses, and delivers educational content efficiently.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Create API endpoints for educational assistance features.
- Establish routes for Question Answering, Quiz Generation, Text Summarization, and Concept Explanation.

```
1  from fastapi import Request
2  from fastapi.staticfiles import StaticFiles
3  from fastapi.templating import Jinja2Templates
4  from explanation_module import explain_topic
5  from quiz_module import generate_quiz
6  from learning_path import get_learning_recommendations
7  from summary_module import summarize_text
8  from fastapi import FastAPI
9  from pydantic import BaseModel
10 from qna import get_answer
11
12 app = FastAPI(
13     title="EduGenie AI Learning Assistant",
14     description="AI Powered Educational Learning Assistant using Gemini and LaMini-Flan-T5",
15     version="1.0.0"
16 )
17
18 app.mount("/static", StaticFiles(directory="static"), name="static")
19
20 templates = Jinja2Templates(directory="templates")
21
22 @app.get("/")
23 async def home(request: Request):
24     return templates.TemplateResponse(
25         request=request,
26         name="index.html"
27     )
28
29 class QuestionRequest(BaseModel):
30     question: str
31
32 class SummaryRequest(BaseModel):
33     text: str
34
35 class LearningPathRequest(BaseModel):
36     topic: str
37
38 class QuizRequest(BaseModel):
39     text: str
40
41 class ExplanationRequest(BaseModel):
```

Process User Requests:

- Accept user queries from the frontend interface.
- Validate and process incoming requests.
- Forward valid requests to the Gemini AI service.

Generate AI Responses:

- Send prompts to Google Gemini API.
- Receive AI-generated educational responses.
- Format outputs for improved readability and user experience

```
34     topic: str
35     @app.post("/qa")
36     def qa(request: QuestionRequest):
37         answer = get_answer(request.question)
38         return {"answer": answer}
39     @app.post("/summary")
40     def summary(request: SummaryRequest):
41         result = summarize_text(request.text)
42         return {"summary": result}
43     @app.post("/learning-path")
44     def learning_path(request: LearningPathRequest):
45
46         result = get_learning_recommendations(
47             request.topic
48         )
49
50         return {"learning_path": result}
51     @app.post("/quiz")
52     def quiz(request: QuizRequest):
53
54         result = generate_quiz(request.text)
55         return {"quiz": result}
56     @app.post("/explain")
57     def explain(request: ExplanationRequest):
58         result = explain_topic(request.topic)
59         return {"explanation": result}
60
61
62     @app.get("/healthz")
63     def healthz():
64         return {"status": "ok"}
```

Error Handling:

- Validate user inputs before processing.
- Handle API communication failures gracefully.
- Display appropriate error messages when required.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly, responsive, and interactive interface for EduGenie. The frontend is designed to provide students with an intuitive learning experience while enabling seamless interaction with AI-powered educational features.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main user interface using HTML.
- Create sections for user input, educational features, and AI-generated results.
- Use semantic HTML elements to improve accessibility and maintainability.
- Design a clean layout suitable for students and learners.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>EduGenie – AI Powered Educational Learning Assistant</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Spectral:wght@500;600;700&family=Inter:wght@400;500;600;700&family=JetBrainsMono:wght@400;500&display=swap"
rel="stylesheet">
<link rel="stylesheet" href="{ url_for('static', path='css/style.css') }" />
</head>
<body>

<div class="app-shell">

<!-- Sidebar -->
<aside class="sidebar">
<div class="brand">
<span class="brand-mark">◆</span>
<div class="brand-text">
<span class="brand-name">EduGenie</span>
<span class="brand-tag">Learning Assistant</span>
</div>
```

Design a Responsive Layout Using CSS:

- Create a responsive user interface using CSS.
- Ensure compatibility across desktop, tablet, and mobile devices.
- Implement modern styling for improved user experience.
- Use flexible layouts and media queries for responsiveness.

```
:root {
  --bg: #0f1320;
  --surface: #161b2e;
  --surface-2: #1e2540;
  --surface-3: #252c4a;
  --border: #2a3152;
  --text: #e8eaf2;
  --text-dim: #9097b3;
  --text-faint: #5c6385;
  --accent: #f2a93b;
  --accent-soft: rgba(242, 169, 59, 0.14);
  --accent-2: #6fe7dd;
  --danger: #f06a6a;
  --danger-soft: rgba(240, 106, 106, 0.12);
  --success: #6fe7a3;

  --font-display: "Spectral", Georgia, serif;
  --font-body: "Inter", -apple-system, BlinkMacSystemFont, sans-serif;
  --font-mono: "JetBrains Mono", Menlo, monospace;

  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 16px;
  --shadow-soft: 0 10px 30px rgba(0, 0, 0, 0.35);
}

* { box-sizing: border-box; }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.001ms !important;
    transition-duration: 0.001ms !important;
  }
}
```

Create Separate UI Components for Each Feature:

- Question Answering Section: Display AI-generated answers to user queries.
- Concept Explanation Section: Present detailed explanations for learning topics.
- Quiz Generation Section: Display generated quiz questions and answers.
- Text Summarization Section: Show concise summaries of educational content.
- Learning Path Section: Provide personalized learning recommendations.

```
qa: {
  title: "Ask a Question",
  eyebrow: "Module 01 - Q&A",
  desc: "Ask EduGenie anything about your syllabus and get a clear, sourced-style answer.",
  endpoint: "/qa",
  buttonLabel: "Get Answer",
  loading: "Genie is thinking...",
  fields: [
    { name: "question", label: "Your question", type: "textarea",
      placeholder: "e.g. What is the difference between TCP and UDP?", required: true }
  ]
},
explain: {
  title: "Explain a Concept",
  eyebrow: "Module 02 - Explanation",
  desc: "Get a step-by-step explanation of any topic, tuned to your level.",
  endpoint: "/explain",
  buttonLabel: "Explain This",
  loading: "Genie is preparing the explanation...",
  fields: [
    { name: "topic", label: "Topic or concept", type: "text",
      placeholder: "e.g. Recursion in programming", required: true },
    { name: "level", label: "Explain for", type: "select",
      options: ["Beginner", "Intermediate", "Advanced"], required: false }
  ]
},
quiz: {
  title: "Generate a Quiz",
  eyebrow: "Module 03 - Quiz Generation",
  desc: "Create a quiz from any text passage.",
```

Activity 4.2: Dynamic Content Rendering Using JavaScript

Handle User Requests Dynamically:

- Capture user input through interactive forms.
- Validate user requests before submission.
- Send requests to the FastAPI backend asynchronously.

Display AI Responses:

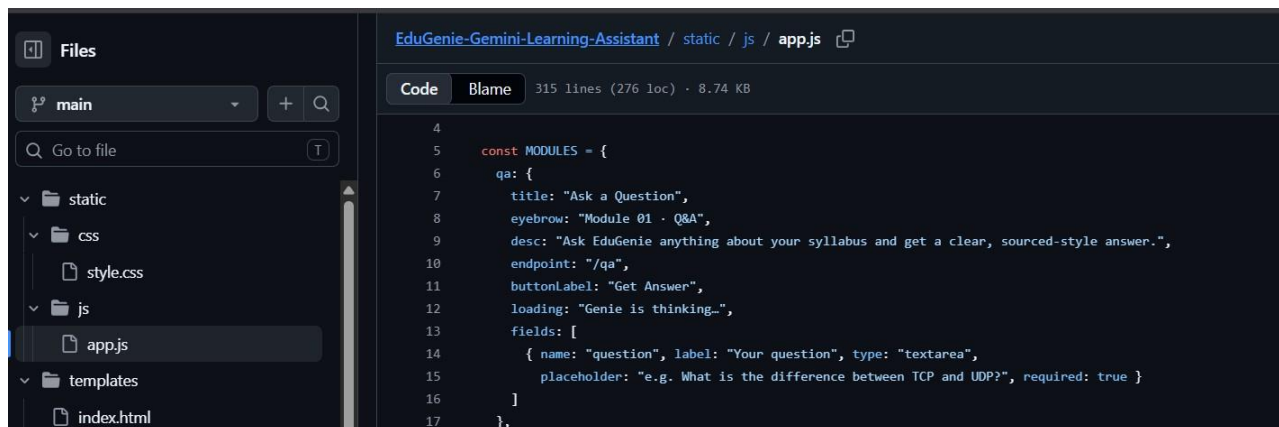
- Render AI-generated responses dynamically without refreshing the page.
- Update the user interface based on returned results.
- Display loading indicators while processing requests.

Improve User Experience:

- Provide instant feedback for user actions.
- Handle errors gracefully and display appropriate messages.
- Ensure smooth interaction between frontend and backend services.

Frontend Technologies Used

- HTML: Structure and layout design.
- CSS: Styling and responsive design.
- JavaScript: Dynamic content rendering and API communication.



The screenshot shows a code editor interface. On the left, a file explorer pane displays the project structure: a 'main' directory containing 'static', 'css', 'js', and 'templates' subdirectories. The 'static/js' directory is selected, showing 'app.js'. The main editor area displays the content of 'app.js', which is a JavaScript module defining a 'qa' object. The object includes a title, eyebrow, description, endpoint, button label, loading text, and a form field configuration. The code is as follows:

```
4
5  const MODULES = {
6    qa: {
7      title: "Ask a Question",
8      eyebrow: "Module 01 - Q&A",
9      desc: "Ask EduGenie anything about your syllabus and get a clear, sourced-style answer.",
10     endpoint: "/qa",
11     buttonLabel: "Get Answer",
12     loading: "Genie is thinking...",
13     fields: [
14       { name: "question", label: "Your question", type: "textarea",
15         placeholder: "e.g. What is the difference between TCP and UDP?", required: true }
16     ]
17   },
```

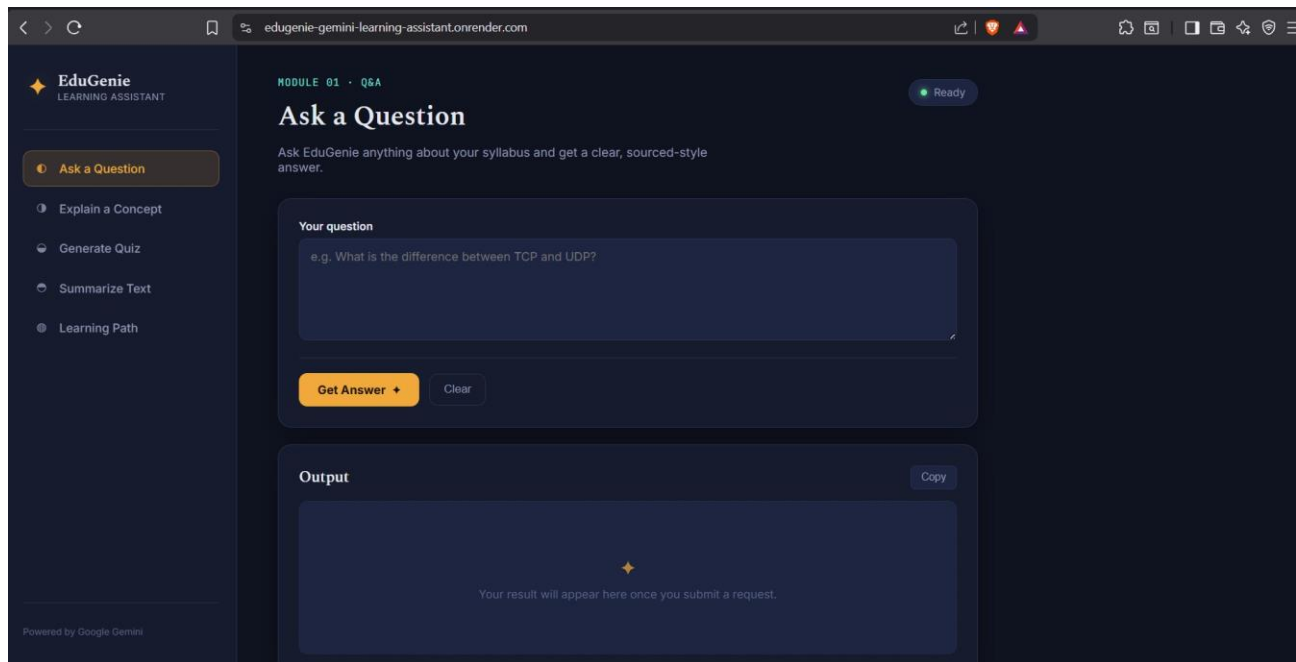
Milestone 5: Deployment

This milestone focuses on testing the EduGenie application, ensuring all features function correctly, and deploying the application for public access. The deployment process makes the system available to students through a secure and scalable hosting platform.

Activity 5.1: Application Testing

Functional Testing

- Verify Question Answering functionality.
- Test Concept Explanation feature.
- Validate Quiz Generation results.
- Verify Text Summarization outputs.
- Test Personalized Learning Path recommendations



User Interface Testing

- Verify responsive design across devices.
- Test navigation and user interactions.
- Ensure proper display of AI-generated responses.

API Testing

- Verify communication between FastAPI and Gemini API.
- Test request and response handling.
- Validate error handling mechanisms.

Activity 5.2: Deployment on Render

Prepare the Application

- Configure project dependencies.
- Verify application configuration files.
- Ensure environment variables are properly configured.

Deploy the Application

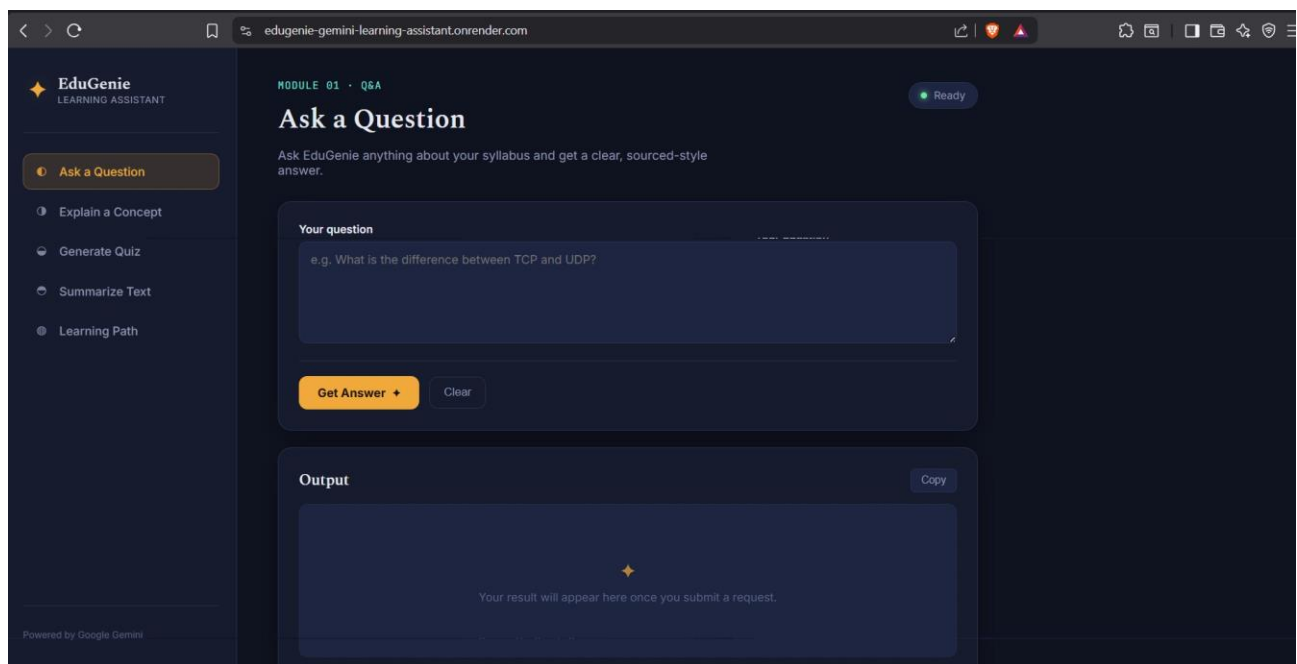
- Upload source code to GitHub repository.
- Connect GitHub repository with Render.
- Configure deployment settings.
- Deploy the FastAPI application.

Verify Deployment

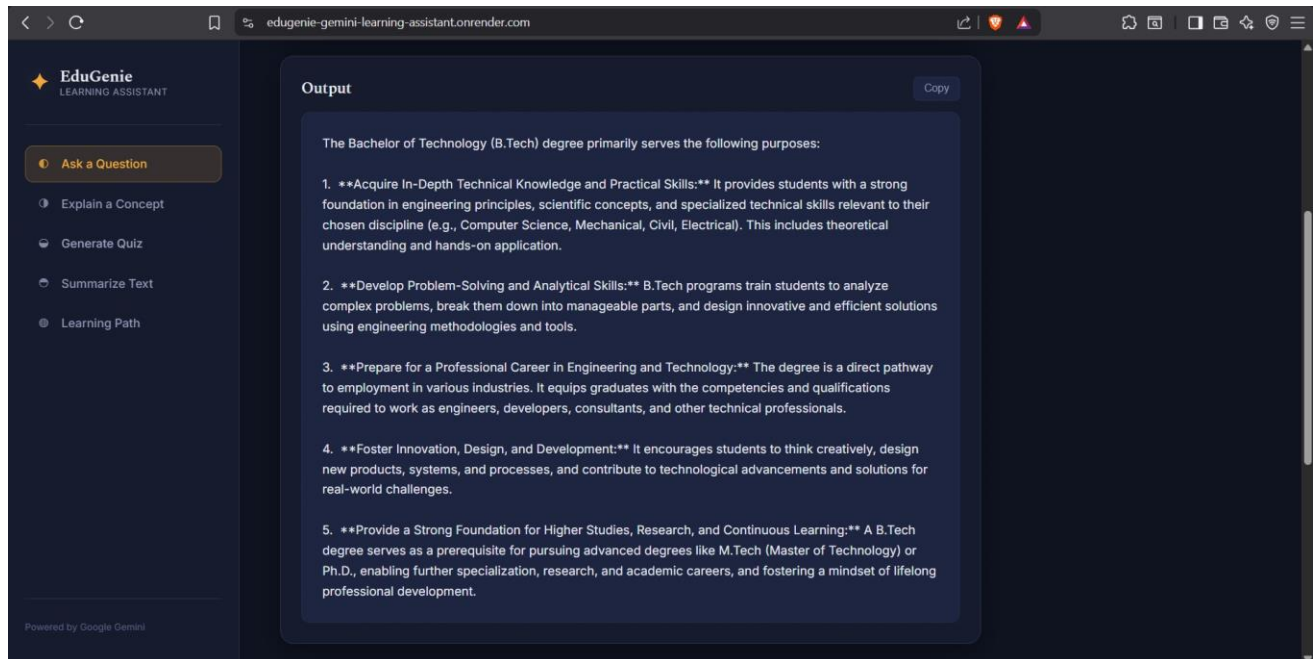
- Access the deployed application.
- Test all major features after deployment.
- Confirm stable application performance.

Deployment Details

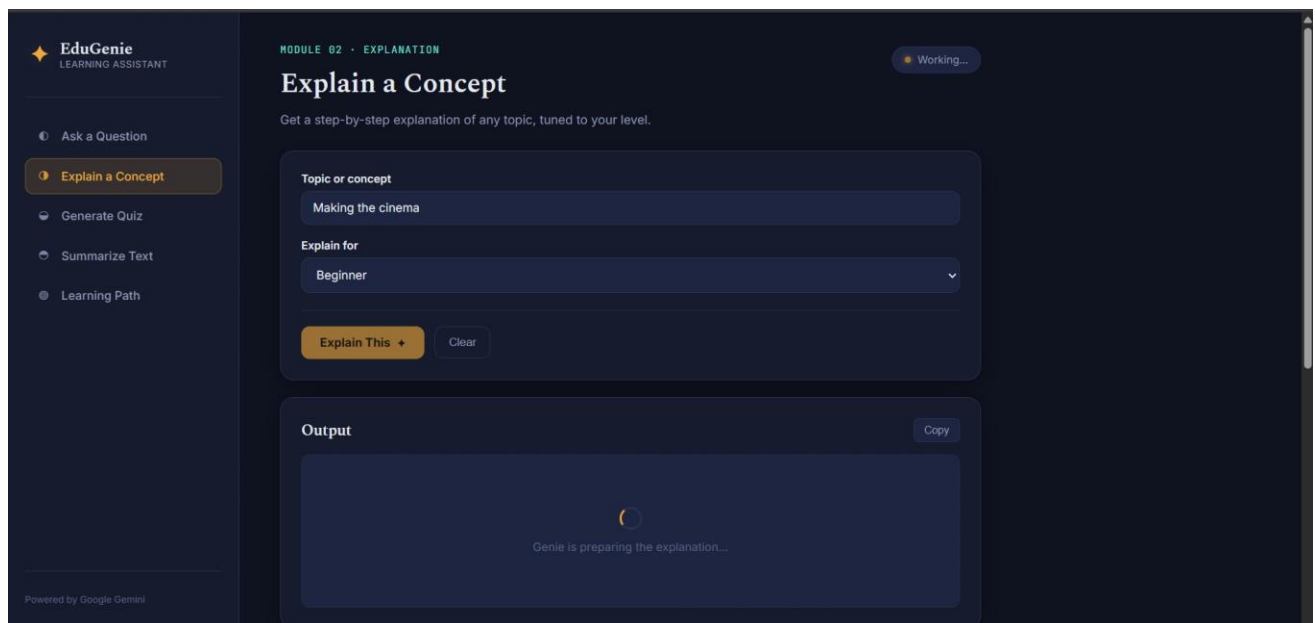
- Hosting Platform: Render
- Backend Framework: FastAPI
- AI Service: Google Gemini API
- Version Control: GitHub



Testing the Module - Ask a question in Edu Genie



The Output of the ask a question module.



Next , Explain A concept


EduGenie
LEARNING ASSISTANT

Ask a Question


Explain a Concept

Generate Quiz

Summarize Text

Learning Path

Powered by Google Gemini

Here's how it generally works, step-by-step:

Making a Cinema (Movie) in Simple Language:

- 1. **It All Starts with an Idea (Story)****
 - **What it is:**** Someone has a cool idea for a story they want to tell. It could be a funny joke, a scary dream, or an adventure.
 - **Example:**** "What if a toy cowboy and a space ranger get lost and have to find their way home?" (That's the idea for Toy Story!)
- 2. **Write It Down (The Script)****
 - **What it is:**** That idea then gets written down into a "script." This is like a detailed recipe for the movie. It tells:
 - What happens
 - Who says what (dialogue)
 - Where everything takes place
 - How the characters feel
 - **Example:**** The script for **Toy Story** would say: "WOODY (sadly) 'Buzz, we're lost.' BUZZ (confidently) 'To infinity and beyond!'"
- 3. **Gather Your Team (Crew & Actors)****
 - **What it is:**** You can't make a movie alone! You need a lot of people:
 - **Director:**** The "captain" of the ship, who guides everyone and makes sure the story comes alive as imagined.
 - **Actors:**** People who pretend to be the characters from the script.
 - **Camera Person (Cinematographer):**** Operates the camera to capture all the shots beautifully.
 - **Sound Person:**** Records all the voices and sounds.
 - **Lighting Person:**** Makes sure everything is lit just right so you can see it clearly and feel the

The Output of Explain A concept

Project URL

[Edu Genie Project URL](#)

Descriptions of Each Module

Ask a Question

The Ask a Question feature enables students to submit academic queries and receive instant AI-generated answers. It helps learners clarify doubts, understand difficult concepts, and access relevant educational information quickly. This feature promotes self-learning and provides personalized academic assistance.

Explain a Concept

The Explain a Concept feature provides detailed and easy-to-understand explanations for educational topics. Students can enter any concept, and the AI generates structured explanations with key points and examples. This helps improve conceptual understanding and supports effective learning.

Generate Quiz

The Generate Quiz feature creates topic-based quizzes to help students assess their knowledge and learning progress. AI-generated questions encourage self-evaluation, strengthen subject understanding, and support exam preparation through interactive learning.

Summarize Text

The Summarize Text feature converts lengthy educational content into concise and meaningful summaries. It highlights important information, reduces reading time, and helps students quickly review and revise study materials.

Learning Path

The Learning Path feature provides personalized learning recommendations based on the student's interests and educational goals. It suggests relevant topics, study sequences, and learning resources, helping students follow a structured and effective learning journey.

Conclusion

EduGenie is an AI-powered educational learning assistant designed to enhance the learning experience through intelligent and personalized educational support. By integrating features such as Ask a Question, Explain a Concept, Generate Quiz, Summarize Text, and Learning Path, the system helps students learn more effectively, improve conceptual understanding, and strengthen academic performance. The successful integration of FastAPI, Google Gemini AI, and modern web technologies demonstrates the potential of Artificial Intelligence in transforming education and supporting students in their learning journey. [\[2\]](#)

